
The Islamia University of Bahawalpur
Department of Computer Science



SOFTWARE DESIGN DESCRIPTION
(SDD DOCUMENT)

for

FixItNow

Version 1.0

By

Muhammad Yar Sanwal

F22BDOCS1M01119

Fall 2022 – 2026

Supervisor

Ms. Iqra Yaqoob

Bachelor of Science in Computer Science

Table of Contents

Introduction	1
Design methodology and software process model	1
System Overview	
1. Presentation Layer	2
2. Application/Logic Layer.....	2
3. Data Layer	2
A) User.....	6
B) Customer	7
C) Service Provider	7
D) Service Category	7
E) Service Listing	7
F) Booking	7
G) Payment	7
H) ChatMessage	7
4.Algorithm & Implementation	8
i) Booking Creation Algorithm (Pseudo-code)	8
ii) Provider Acceptance Algorithm	9
Software requirements traceability matrix	9

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Ms. Iqra Yaqoob

Signature _____

Introduction

FixItNow is an on-demand home services platform designed to connect customers with professional service providers for tasks such as plumbing, electrical repair, cleaning, painting, and other home services. This SDD document describes the **design decisions, architecture, data modeling, diagrams, and algorithms** used in the system. Modules completed till now include:

1. User Management (Customer & Service Provider)
2. Service Browsing
3. Booking Module
4. Notification & Chat Module
5. Admin Management Module

Design methodology and software process model

Design Methodology

FixItNow follows an **Object-Oriented Design (OOD)** methodology because:

1. The system naturally fits into objects like User, Provider, Booking, Service Category, etc.
2. OOP promotes modularity, reusability, and flexibility. UML modelling aligns directly with OOP design.

Software Process Model

The **Incremental Software Development Model** is used because:

1. The system has independent modules that can be built in increments (User Module → Booking → Chat → Admin).
2. Early delivery of core functionalities is possible.
3. Changes based on supervisor feedback can be incorporated easily. Reads project risk through stepwise improvements.

System Overview

FixItNow follows a **three-role system**:

1. **Customer:** Browses services, books providers, makes payments, chats, rates providers.
2. **Service Provider:** Accepts bookings, updates status, manages profile, communicates with customers.
3. **Admin:** Manages service categories, approves providers, monitors bookings and system data.
4. The system includes:
 5. Web-based frontend (Next.js)
 6. Mobile app (React Native)
 7. Backend API (Node.js)
 8. Database (Supabase PostgreSQL)

Architectural design

Fixit Now follows a **3 tier** Structure:

1. Presentation Layer

- a. Web: Next.js
- b. Mobile: React Native
 - i. Handles UI, forms, screens, and user interactions.

2. Application/Logic Layer

- a. Node.js API
- b. Booking Logic
- c. Notification Logic
- d. Logic

3. Data Layer

- a. Supabase PostgreSQL
- b. Supabase Storage for media
- c. Authentication & Real-time services

Process flow/Representation

i) Major Process: Service Booking Flow

You will draw an activity diagram based on this flow:

- 1. Customer selects a service category
- 2. System shows available providers
- 3. Customer submits booking request
- 4. Provider receives notification
- 5. Provider Accepts or Rejects
- 6. If accepted → booking confirmed
- 7. Service is completed
- 8. Customer rates provider

Design models

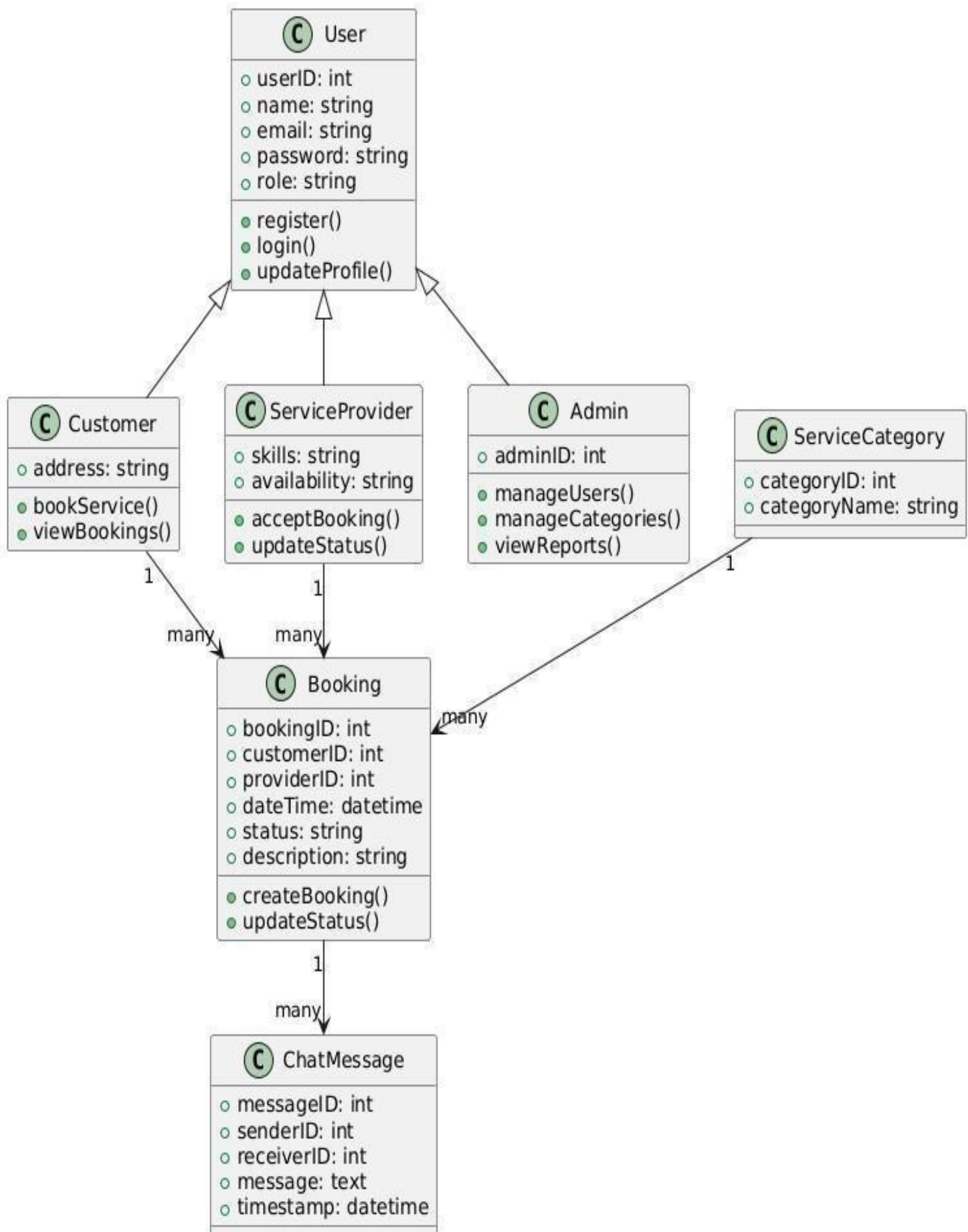
1. CLASS DIAGRAM:

The class diagram represents the object-oriented design structure of FixItNow.

The main entities include **User**, **Customer**, **ServiceProvider**, **Booking**, **ServiceCategory**, **ChatMessage**, and **Admin**.

Inheritance is used to differentiate between user roles. Associations show booking, communication, and service relationships.

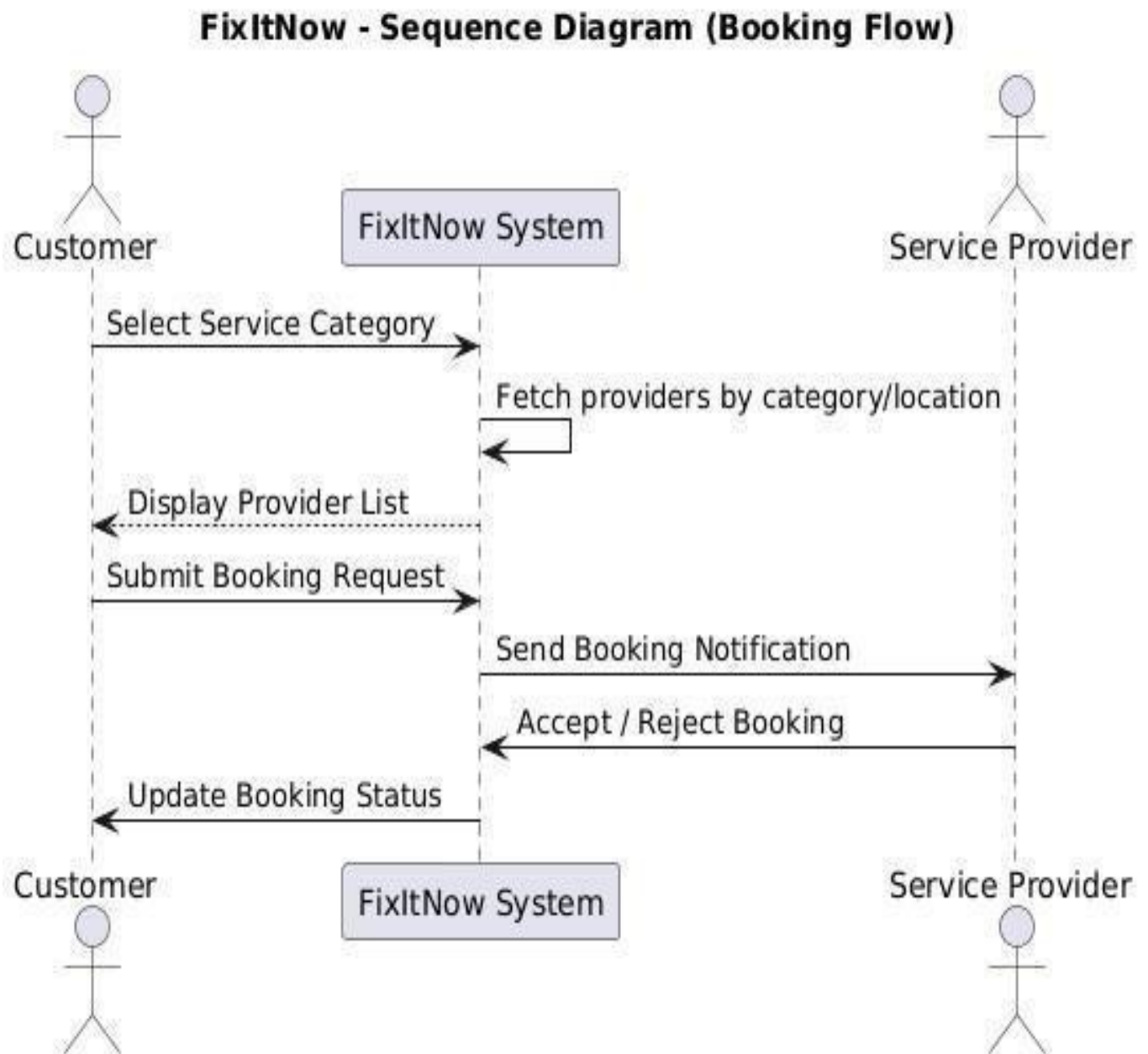
FixItNow - Class Diagram



2. SEQUENCE DIAGRAM:

The sequence diagram describes the interaction flow for the **Service Booking Process**.

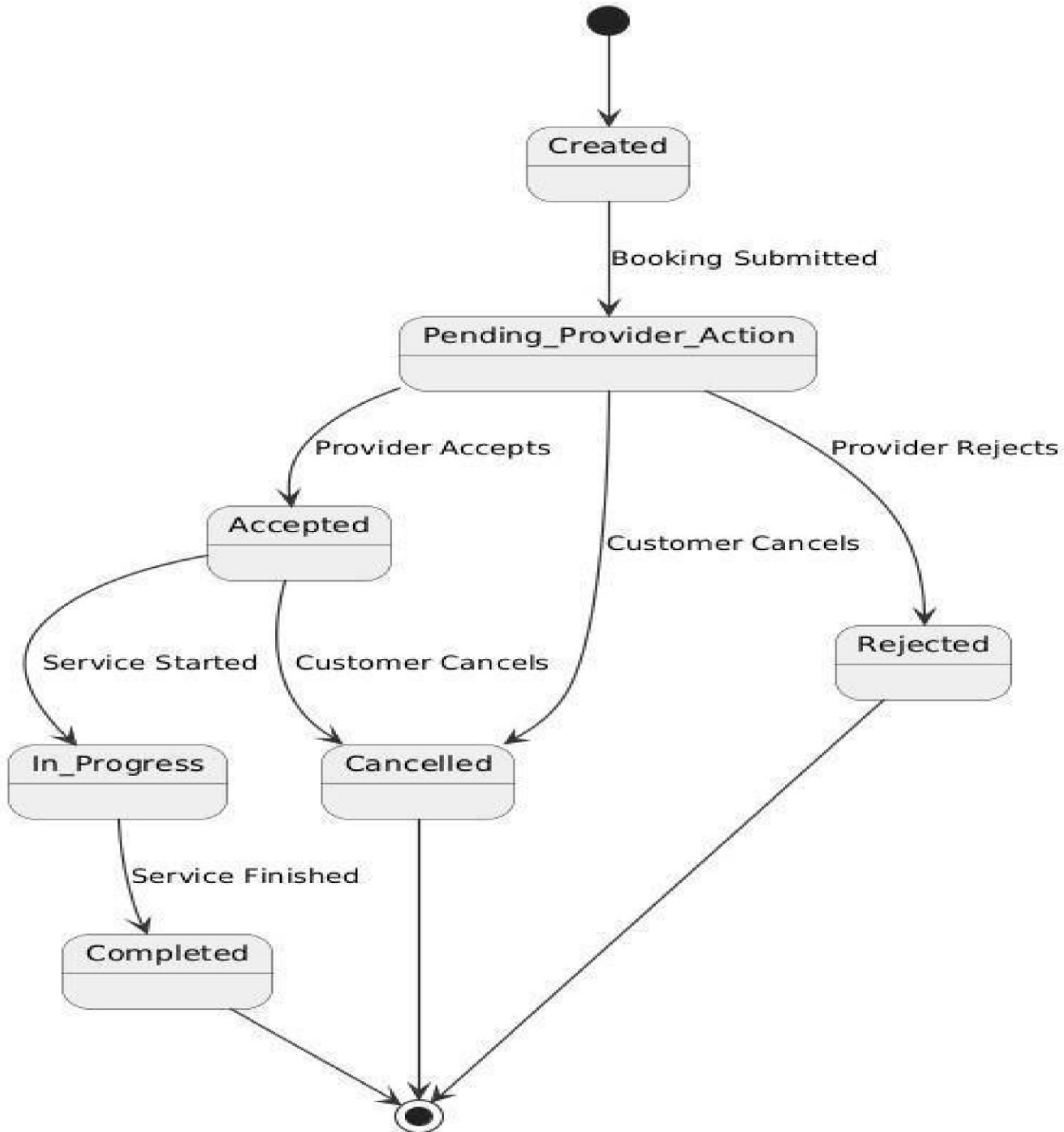
It shows chronological message exchanges between the Customer, System, and Service Provider. This diagram helps explain how a booking is created, processed, and accepted.



3. STATE TRANSITION DIAGRAM:

The state transition diagram represents the various **states of a booking** throughout its lifecycle in FixItNow. It helps visualize how a booking moves from creation to completion, including cancellation or rejection. This is essential for understanding system flow and backend logic.

FixItNow - Booking State Transition Diagram



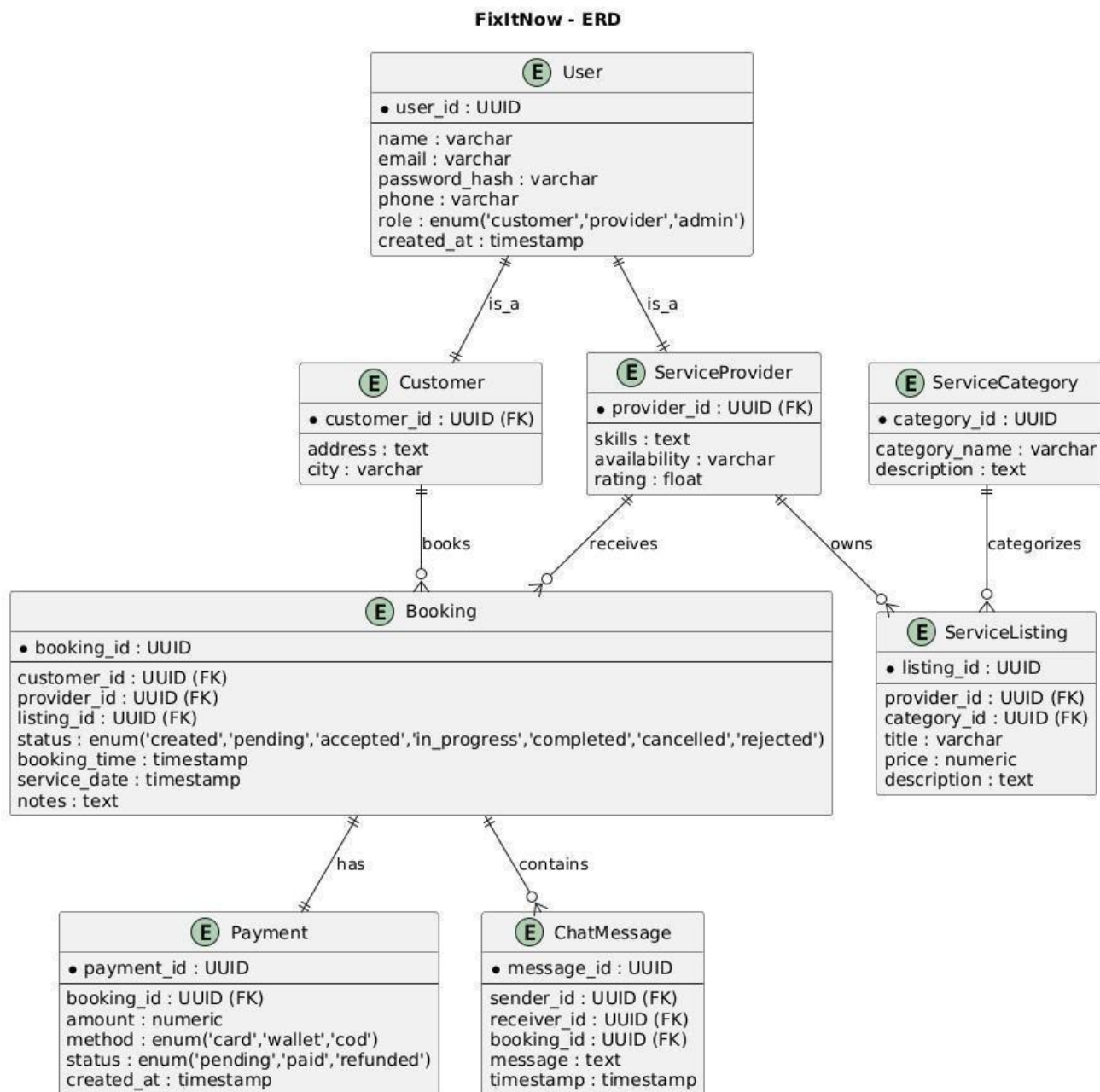
4. ENTITY RELATIONSHIP DIAGRAM:

- A) **User:** Central authentication entity storing common fields for customers, service providers, and administrators.
- B) **Customer:** Extends User; stores customer-specific data like address and city.
- C) **Service Provider:** Extends User; stores provider skills, availability, and rating.
- D) **Service Category:** Defines high-level service categories (Plumbing, Electrical, Furniture Repair etc.).
- E) **Service Listing:** Services offered by providers under specific categories.

F) Booking: Represents customer requests. Tracks status, timestamps, and related provider+listing. G)

G) Payment: Stores payment records for bookings.

H) ChatMessage: Stores communication between customer and provider per booking.



Algorithm & Implementation

Provide pseudo-code for main modules.

i) Booking Creation Algorithm (Pseudo-code)

FUNCTION createBooking(customerID, providerID, description, dateTime):

 booking.status = "Pending"

 STORE booking IN database

 SEND notification TO provider RETURN bookingID

ii) Provider Acceptance Algorithm

```
FUNCTION respondToBooking(bookingID, decision):  
  IF decision == "Accept":  
    UPDATE booking.status = "Accepted"  
ELSE:  
  UPDATE booking.status = "Rejected"  
SEND notification TO customer
```

Software requirements traceability matrix

This section should contain a table that summarizes how each software requirement has been met in this document. The tabular format permits one-to-one and one-to-many relationships to be shown.

Req. Number	Ref. Item	Design Component	Component Items
FR01	Class Diagram	User Class	Register(), Login()
FR03	Sequence Diagram	Booking Flow	CreateBooking()
FR05	Class Diagram	Provider Class	acceptBooking()
FR08	Sequence Diagram	Chat Flow	sendMesage()

